

CI/CD — Tests, Artifact Promotion and Safe Releases

From a commit to production through gates, one digest, and a canary — with a person still holding the last door

WHAT WE'LL COVER

Four chapters, four labs

01

What a pipeline is for

02

Build once, promote the artifact

03

Environment promotion

04

Release strategies

01

What a pipeline is for

Repeatability, evidence and speed — and the gates, in order, that turn a folder of code into something you can trust.

01 What a pipeline is for

02 Build once, promote the artifact

03 Environment promotion

04 Release strategies

WHAT A PIPELINE IS FOR

Repeatability, evidence and speed — and you are buying them in that order

Repeatability

- same steps, same order, every time
- clean workspace — no 'works on my machine'
- the deploy command is in a file, reviewed

Evidence

- build id, commit, who triggered it
- which gates ran and what they said
- the exact artifact — a digest, not a name

Speed

- ship many times a day safely
- a failure costs minutes, not a release
- only worth it once the first two hold

WORKED EXAMPLE — WHAT ONE BUILD LEAVES BEHIND: A RECORD YOU CAN HAND TO AN AUDITOR

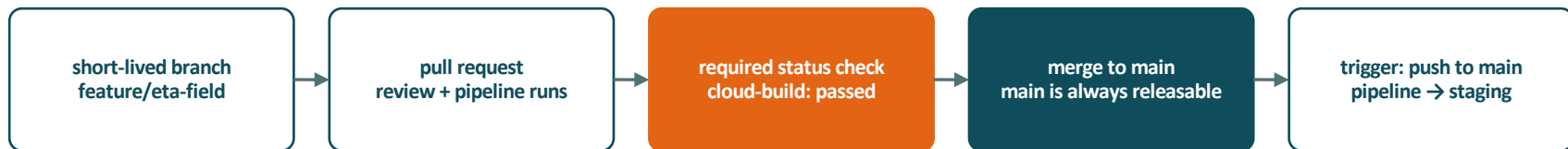
```
$ gcloud builds describe 2b18...aada --format='yaml(id,status,substitutions,steps[].id,steps[].status,results.images[].digest)'  
id: 2b1824b6-0f8a-4f0a-853e-f265d278aada      status: SUCCESS      substitutions: { _SHA: 9f3c2e1}  
steps: [lint OK, typecheck OK, unit-tests OK, contract-tests OK, dependency-scan OK, build OK, integration OK, push OK, vulnerability-  
scan OK]  
results.images[0].digest: sha256:ab12... # the one thing that goes to production
```

THE RULE

A pipeline is the deploy written down and run by a machine — repeatable first, recorded second, fast third.

TRUNK-BASED DEVELOPMENT AND BRANCH PROTECTION

One branch that is always releasable — and nobody pushes to it directly



BRANCH PROTECTION ON MAIN

- no direct pushes — pull requests only
- required status check: the pipeline must pass
- at least one review; dismiss stale approvals on new commits
- the same rule for admins

THE ANTI-PATTERN

- long-lived feature branches that merge monthly
- the pipeline tested the branch, production got the merge
- 'we'll fix the tests after the release'

THE RULE

Main is always releasable because nothing reaches it except through the pipeline — that is what branch protection enforces.

THE TEST PYRAMID FOR A SERVICE

Many fast unit tests, a few contract tests, a handful through the real thing

End-to-end

a request through the deployed system

Contract

the api produces what the web page consumes

Integration

the built container answers /healthz

Unit

one endpoint, no network, milliseconds

```
# tests/contract/test_contract.py - what the web page reads from the api
WEB_EXPECTS_ON_ROOT = {"version"}
WEB_EXPECTS_ON_BOOK = {"title", "author", "year"}
def test_each_book_has_the_fields_the_web_page_reads():
    for book in api.app.test_client().get("/books").get_json():
        assert WEB_EXPECTS_ON_BOOK <= set(book), f"missing {WEB_EXPECTS_ON_BOOK - set(book)}"
```

THE RULE

Test the shape between services, not just the logic inside them — the contract test is the one that protects the neighbour.

RUNNING TESTS IN A PIPELINE, AND WHAT TO DO WHEN THEY ARE SLOW

Fastest gates first, one failure stops everything, slow suites move down the pyramid

Step	What it does	Typical time	Fails on
tooling	one venv, pinned tools, shared by every gate	40 s	a broken pin
lint · typecheck	ruff, mypy	5 s · 10 s	style, unused imports, type errors
unit · contract	pytest tests/unit, tests/contract	3 s · 2 s	logic; the shape between services
dependency-scan	pip-audit against pinned requirements	15 s	a known CVE in a dependency
build	docker build, tagged with the commit	2–3 min	Dockerfile errors
integration	run the image, curl /healthz	10 s	wrong port, missing package
push · vulnerability-scan	push; on-demand scan; fail on CRITICAL	30 s · 60 s	a critical CVE in the base image

Speed levers: order (cheap first) · parallel steps (waitFor) · a cached toolchain · sharding · moving slow tests down a layer.

THE RULE

Order gates by cost — a failure should be reported by the cheapest step that can detect it.

LINTING, TYPE CHECKING AND DEPENDENCY SCANNING AS GATES

Three checks that need no test to be written — and catch a class of bug each

WORKED EXAMPLE — THE THREE GATES, EACH FAILING ON SOMETHING A TEST WOULD NOT HAVE CAUGHT

```
$ ruff check services tests
services/bookshelf-api/app.py:14:1: F401 [*] `socket` imported but unused
Found 1 error.
$ mypy services/bookshelf-web/app.py
app.py:41: error: Argument 1 to "fetch_id_token" has incompatible type "None";
expected "Request" [arg-type]
$ pip-audit --strict -r services/bookshelf-web/requirements.txt
Found 1 known vulnerability in 1 package
Name      Version ID                               Fix Versions
requests  2.31.0  GHSA-9wx4-h78v-vm56 2.32.0
```

What each gate is for

- Lint: dead code, undefined names, one style — reviews stay about logic
- Types: the None and the wrong argument, before runtime
- Dependencies: a published CVE in a pinned version — no code change needed to fail
- All three pinned in requirements-dev.txt, all three in seconds

THE RULE

Static gates are tests nobody had to write — lint, types and a CVE database, in seconds, before anything builds.

LAB 1

Run the pipeline, break a gate, read the timings, let the contract test catch a rename

LAB 1 · LAB MANUAL

Open the lab manual, Lab 1. Every command, console path and expected output is there — nothing is typed from this slide.

- Enable APIs, create a local git history, submit the pipeline: all twelve steps pass, an image tagged with the commit is pushed
- Change an expected value in a unit test, commit, resubmit — the build stops at unit-tests, no image exists for that commit
- Read the step timings of both builds; see why cheap gates run first
- Rename a field the web page reads; the contract test fails before anything is built

About 25 minutes
Kit: gcp-cicd-labs.zip
Cloud Shell only — no code

THE RULE

Twelve gates, one failure, one contract — the pipeline stopped where it should and said why.

02

Build once, promote the artifact

One build, one digest, immutable tags, a scan you can fail on, and an identity for the pipeline that holds no keys.

01 What a pipeline is for

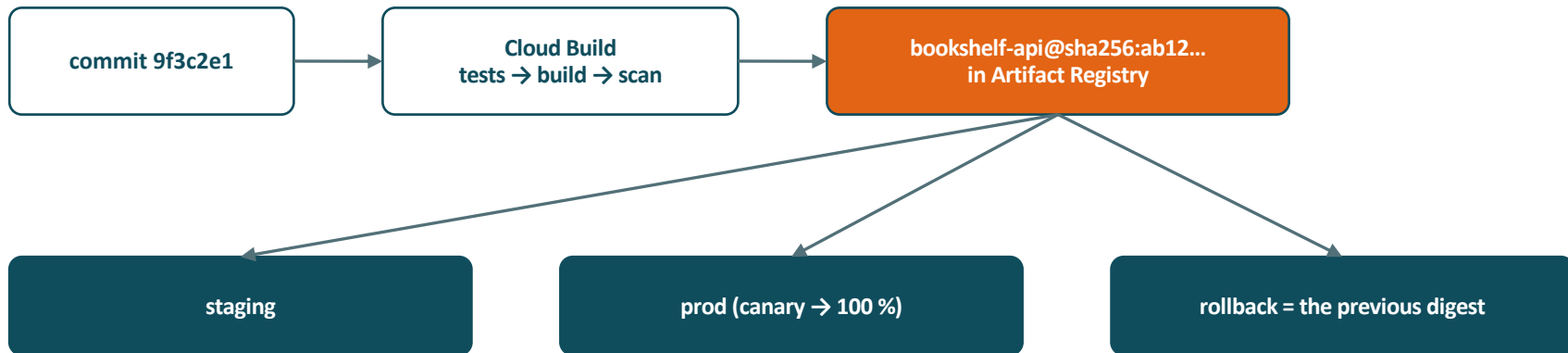
02 Build once, promote the artifact

03 Environment promotion

04 Release strategies

BUILD THE IMAGE ONCE AND PROMOTE THE SAME ARTIFACT

What staging tested is what production runs — because it is the same bytes



Identical across environments	Different per environment
the image digest	service account · secrets · env vars · ingress · scaling limits

THE RULE Build once, promote the digest — a rebuild per environment means staging proved nothing.

IMMUTABLE TAGS, AND WHY 'LATEST' IS A DEFECT

A digest is a fact; a tag is a pointer — and 'latest' is a pointer others move

WORKED EXAMPLE — ONE DIGEST, MOVING TAGS

```
$ gcloud artifacts docker images list $AR --include-tags --format='table(tags,version)'  
TAGS      VERSION  
9f3c2e1   sha256:ab12c3...   # commit tag → this digest, forever  
$ gcloud artifacts docker tags add $AR/bookshelf-api:9f3c2e1 $AR/bookshelf-api:latest  
# ... a second build pushes commit e7d09a4 and someone tags it latest too ...  
TAGS      VERSION  
9f3c2e1   sha256:ab12c3...  
e7d09a4, latest  sha256:77f0e1...   # 'latest' now means something else; the old deploy command is a lie  
$ gcloud run deploy orders-api --image $AR/bookshelf-api@sha256:ab12c3...   # deploy by digest: unambiguous, rollback-able
```

THE RULE SET

- tag = commit SHA · deploy = digest · latest = never created

WHAT LATEST COSTS

- no reproducibility, no rollback target, no evidence of what ran

THE RULE

Tag by commit for people, deploy by digest for machines, and never create a tag called latest.

ARTIFACT REGISTRY, PROVENANCE AND VULNERABILITY SCANNING

The registry scans what you push and records who built it — the pipeline fails on findings

WORKED EXAMPLE — SCAN, COUNT, GATE; PROVENANCE FROM THE BUILD

```
$ SCAN=$(gcloud artifacts docker images scan $AR/bookshelf-api:9f3c2e1 --remote --format='value(response.scan)')
$ gcloud artifacts docker images list-vulnerabilities $SCAN --format='value(vulnerability.effectiveSeverity)' | sort | uniq -c
  2 HIGH
 11 LOW
   6 MEDIUM
# the gate, from cloudbuild.yaml:
N=$(... | grep -c '^CRITICAL$'); if [ "$N" -gt 0 ]; then echo "GATE FAILED: $N CRITICAL"; exit 1; fi
$ gcloud artifacts docker images describe $AR/bookshelf-api:9f3c2e1 --show-provenance | head -3
provenance_summary: ... builder: https://cloudbuild.googleapis.com/GoogleHostedWorker  slsa_level: 2
```

Automatic scan	On-demand scan	Provenance
every push, results per digest, viewable in the console	in the pipeline, right after push — the gate reads it	signed record of build id, commit, config; SLSA level

THE RULE

Scan after push and fail on a severity you chose on purpose — and keep the provenance that proves where the image came from.

PIPELINE IDENTITY AND KEYLESS AUTHENTICATION

The pipeline is a workload — its own service account, actAs on the deployers, no key

Role	Granted on	Lets the pipeline
roles/clouddeploy.jobRunner	project	render manifests and run rollouts
roles/run.developer	project	create and update Cloud Run revisions
roles/iam.serviceAccountUser	bookshelf-api-min (each runtime identity)	deploy a service that runs as that identity — and no other
roles/artifactregistry.reader	the registry (shared project in real layouts)	pull the digest it promotes

```
# deploy/clouddeploy.yaml — both targets run as the pipeline identity, not the default compute SA
executionConfigs:
- usages: [RENDER, DEPLOY]
  serviceAccount: bookshelf-deployer@PROJECT_ID.iam.gserviceaccount.com
```

THE RULE

The pipeline is a workload: its own identity, roles for its job, actAs on the deployers it serves — and never a key.

Digest, tag, scan — and 'latest' refused

LAB 2 · LAB MANUAL

Open the lab manual, Lab 2. Every command, console path and expected output is there — nothing is typed from this slide.

- Read the image the good build pushed: commit tag and digest are two names for one thing
- Run the on-demand scan by hand and count findings by severity; see what lowering the gate to HIGH would do
- Create a latest tag, watch what it points at, and see why Cloud Deploy will not take it — then delete it
- Read the build provenance attached to the image

About 20 minutes
Kit: gcp-cicd-labs.zip
Cloud Shell only — no code

THE RULE

One digest, a scan you can fail on, a tag you never create — the artifact is now something you can promote.

03

Environment promotion

The same digest moves through staging to production; only the surroundings change, and a person approves the last step.

- 01 What a pipeline is for
- 02 Build once, promote the artifact
- 03 Environment promotion**
- 04 Release strategies

ENVIRONMENT PROMOTION

Development to staging to production is one digest moving — never three builds

STAGING

```
# deploy/run-service-staging.yaml
metadata: { name: bookshelf-api-staging, annotations: {
  run.googleapis.com/ingress: all } }
spec.template.spec:
  serviceAccountName: bookshelf-api-min@...
  containers: [ { image: bookshelf-api,          # placeholder
    env: [APP_VERSION=staging, DB_PASSWORD ← secret db-
password:2] } ]
  volumes: [ db-password → /secrets ]
```

PROD

```
# deploy/run-service-prod.yaml
metadata: { name: bookshelf-api, annotations: {
  run.googleapis.com/ingress: internal } }
spec.template.spec:
  serviceAccountName: bookshelf-api-min@...
  containers: [ { image: bookshelf-api,          # same placeholder
    env: [APP_VERSION=prod, DB_PASSWORD ← secret db-password:2]
  } ]
  volumes: [ db-password → /secrets ]
```

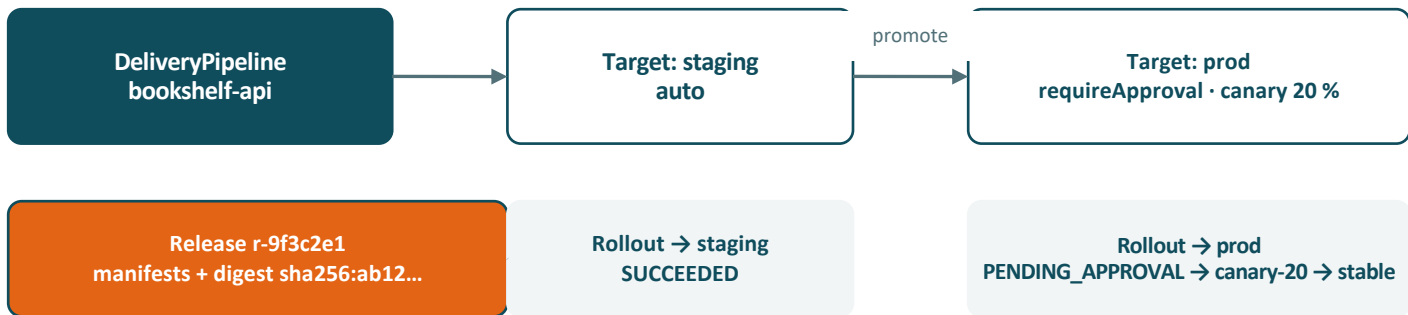
```
$ gcloud deploy releases create r-9f3c2e1 --delivery-pipeline=bookshelf-api --region=asia-south1 --source=deploy \
  --images=bookshelf-api=asia-south1-docker.pkg.dev/$PROJECT/bookshelf/bookshelf-api@sha256:ab12c3... # one digest into both manifests
```

THE RULE

A promotion substitutes one digest into the next environment's manifest — the artifact is proven, the surroundings are swapped.

CLOUD DEPLOY

Pipeline = targets in order; release = one digest; rollout = one release onto one target



Do	Command
register pipeline + targets	<code>gcloud deploy apply --file=deploy/clouddeploy.yaml --region=\$R</code>
create a release (one digest)	<code>gcloud deploy releases create r-\$SHA --delivery-pipeline=bookshelf-api --source=deploy --images=bookshelf-api=\$AR/bookshelf-api@\$DIGEST</code>
promote to the next target	<code>gcloud deploy releases promote --release=r-\$SHA --delivery-pipeline=bookshelf-api</code>
approve · advance · roll back	<code>gcloud deploy rollouts approve ... · rollouts advance ... · targets rollback prod ...</code>

THE RULE

Pipeline = the road, target = a stop, release = the cargo, rollout = one delivery — every command names one of them.

APPROVAL GATES AND WHO HOLDS THEM

Automatic into staging, a named approver into production — the approval is a record

WORKED EXAMPLE — A ROLLOUT WAITING, THE APPROVAL, AND THE RECORD IT LEAVES

```
$ gcloud deploy rollouts list --release=r-9f3c2e1 --delivery-pipeline=bookshelf-api --region=$R --
format='table(name.basename(),targetId,state,approvalState) '
NAME                                TARGET_ID  STATE      APPROVAL_STATE
r-9f3c2e1-to-staging-0001          staging     SUCCEEDED  DOES_NOT_NEED_APPROVAL
r-9f3c2e1-to-prod-0001             prod       PENDING_APPROVAL  NEEDS_APPROVAL
$ gcloud deploy rollouts approve r-9f3c2e1-to-prod-0001 --release=r-9f3c2e1 --delivery-pipeline=bookshelf-api --region=$R
# Admin Activity audit log, always on:
clouddeploy.googleapis.com ApproveRollout principal: priya@example.com rollout: r-9f3c2e1-to-prod-0001 2026-09-09T11:02:14Z
```

Who	Holds	May
group:orders-release-approvers	roles/clouddeploy.approver	approve or reject prod rollouts
bookshelf-deployer (the pipeline)	jobRunner, run.developer, actAs	render and deploy — never approve
group:orders-developers	roles/clouddeploy.releaser	create releases and promote to staging

THE RULE Approval is a role held by people, never by the pipeline — and every approval is already a change record.

LAB 3

Register the pipeline; release to staging, then to prod behind your approval

LAB 3 · LAB MANUAL

Open the lab manual, Lab 3. Every command, console path and expected output is there — nothing is typed from this slide.

- Grant the pipeline identity its two roles; fill the project id into the manifests; register pipeline and targets
- Create a release from Lab 2's digest — staging deploys on its own; call it
- Promote to prod; watch it wait; approve it; advance the 20 % canary to 100 %
- Prove staging and prod run the identical digest, and prod kept Session C's identity, secret and ingress

About 25 minutes
Kit: gcp-cicd-labs.zip
Cloud Shell only — no code

THE RULE

Same digest in two places, one approval in the log — the promotion is a fact, not a hope.

04

Release strategies

Rolling, blue-green and canary; a 20 % canary on Cloud Run driven by the pipeline; rollback as an ordinary release.

- 01 What a pipeline is for
- 02 Build once, promote the artifact
- 03 Environment promotion
- 04 Release strategies**

RELEASE STRATEGIES

Rolling, blue-green and canary — what each costs and what each protects you from

Strategy	How	Costs	Protects from	On Cloud Run
Rolling	replace a few instances at a time	nothing extra; two versions overlap	downtime during deploy	the default: a new revision takes 100 % when ready
Blue-green	full second copy; flip all traffic; keep the old	double capacity while both exist	a bad cut-over — flip back in seconds	two revisions, traffic 100/0 → 0/100, old kept
Canary	a small share of real traffic first; watch; advance or abort	a window with two versions answering	a bad release reaching everyone	traffic 80/20 → 100 — Cloud Deploy automates it

All three assume you can tell good from bad while it happens — that is what the observability session's dashboards and alerts are for.

THE RULE

Pick the strategy by the cost of being wrong: canary when real traffic is the only real test, blue-green when the flip must be instant.

TRAFFIC SPLITTING ON CLOUD RUN AS A RELEASE MECHANISM

Cloud Deploy drives the split: 20 % to the new revision, hold, then advance to 100 %

WORKED EXAMPLE — A CANARY PAUSED AT 20 %, THEN ADVANCED

```
# deploy/clouddeploy.yaml - the prod stage
strategy: { canary: { runtimeConfig: { cloudRun: { automaticTrafficControl: true } }, canaryDeployment: { percentages: [20], verify: false } } }
$ gcloud deploy rollouts describe r-9f3c2e1-to-prod-0001 --release=r-9f3c2e1 --delivery-pipeline=bookshelf-api --region=$R --format='yaml(state,phases[].id,phases[].state)'
state: IN_PROGRESS
phases: [ {id: canary-20, state: SUCCEEDED}, {id: stable, state: PENDING} ]
$ gcloud run services describe bookshelf-api --region $R --format='value(status.traffic)'
{'percent': 80, 'revisionName': 'bookshelf-api-00012-abc'} {'percent': 20, 'revisionName': 'bookshelf-api-00013-def'}
$ gcloud deploy rollouts advance r-9f3c2e1-to-prod-0001 --release=r-9f3c2e1 --delivery-pipeline=bookshelf-api --region=$R
```

- 1 deploy the new revision, 0 % traffic
- 2 shift 20 %; pause in canary-20
- 3 advance → stable: 100 % · or roll back

THE RULE

The manifest never sets traffic — the pipeline does, in phases you advance on evidence.

ROLLBACK AS A FIRST-CLASS PATH, NOT AN EMERGENCY

A rollback is a rollout of the previous release — same tool, same record, no typing

WORKED EXAMPLE — ROLLBACK IS A ROLLOUT

```
$ gcloud deploy targets rollback prod --delivery-pipeline=bookshelf-api --region=$R
Rolling back target prod to release r-9f3c2e1 ...
$ gcloud deploy rollouts list --delivery-pipeline=bookshelf-api --region=$R --filter='targetId=prod' --
format='table(name.basename(),state)'
```

NAME	STATE	
r-9f3c2e1-to-prod-0002	SUCCEEDED	# the rollback: the previous release, deployed again
r-bad-to-prod-0001	CANCELLED	
r-9f3c2e1-to-prod-0001	SUCCEEDED	

WHY IT IS SAFE

- the previous release is a digest that still exists
- the same pipeline identity and manifests deploy it
- it leaves the same record as any rollout

WHAT IT DOES NOT UNDO

- a schema migration · a rotated secret · data the bad version wrote — each rolls back separately (Session B)

THE RULE

Practise rollback on a good day so it is ordinary on a bad one — it is one command because every release is a digest.

DATABASE AND SCHEMA CHANGE ALONGSIDE A CODE RELEASE

Expand, migrate, contract — the schema goes first and never rolls back with the code

Step	Change	Old code	New code	Rollback-safe?
1 • Expand	add column `name` nullable; keep `title`	works	works	yes — nothing removed
2 • Migrate	backfill name = title (background job)	works	works	yes
3 • Code release	new code writes both, reads name; canary → 100 %	—	works	yes — old code still finds title
4 • Contract	drop `title` — weeks later, its own release	gone	works	no — that is why it is last and separate

Rule of thumb: a schema change and a code change never share a release. The schema is always one step ahead, and always backward-compatible with the code that might come back.

THE RULE

Schema first, code second, cleanup last — and never in the same release, so the code can always come back.

LAB 4

A bad release meets 20 % of prod — see it, then roll back through the pipeline

LAB 4 · LAB MANUAL

Open the lab manual, Lab 4. Every command, console path and expected output is there — nothing is typed from this slide.

- Baseline: twenty page loads, zero failures
- Release the same digest with a faulty setting; promote, approve — the canary takes 20 %
- Watch roughly one load in ten fail through bookshelf-web; find the fault lines in the logs; do not advance
- Roll back with one command; confirm 100 % on the good revision; then fix forward the normal way

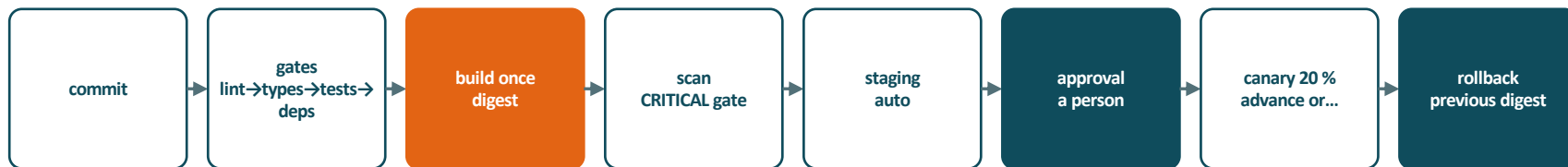
About 25 minutes
Kit: gcp-cicd-labs.zip
Cloud Shell only — no code

THE RULE

A bad release met a fifth of your users, not all of them — and left through the same door it came in.

THE WHOLE PATH IN ONE PICTURE

commit → gates → digest → scan → staging → approval → canary → prod → rollback



You leave with	
a pipeline that stops at the cheapest failing gate	and says which one, in the record
one digest per commit, never a latest	promoted, not rebuilt
a human approval and a canary on prod	both recorded, both reversible
rollback as one ordinary command	practised today, so it works on the day it matters

The service now reaches production through a door with a record. Next: watching it, then breaking it.

Observability turns the canary's 'watch the page' into dashboards, SLOs and alerts. Incident response starts from an alert on this pipeline's prod service — and rolls back with the command you ran today.

Leave the pipeline and both services running. Kit: [gcp-cicd-labs.zip](#) · Lab manual: [gcp-cicd-lab-manual.pdf](#)